Semantic Block Codes

Distilling a Sentence Encoder into a qFHRR Word Table for Microsecond Retrieval and Routing

Nicolas Cloutier, GrillCheese Research Lab (Lévis, Québec)

Preprint v1, November 2026. Working paper of August 2026, with results added through October 2026.

Abstract

Vector symbolic architectures built on Fourier holographic reduced representations (FHRR) give a complete algebra, with binding, bundling and exact unbinding, but their vectors are random by construction, so two texts that mean the same thing get orthogonal codes. Sentence encoders have the opposite profile: they carry semantic geometry, but they cost milliseconds per call and they have no algebra. We show the two are not in tension. We distill a sentence encoder into a static word table over sparse block codes, the quantized-phase form of FHRR (qFHRR), through a cosine-exact orthonormal projection blended 0.5/0.5 with deterministic random phasor identities. The result is a text encoder that is a pure table lookup with no model in the serving path, about 7 to 25 µs per short text on one CPU core, and it keeps the block structure that binding and bundling operate on. On a 15-domain routing screen the table reaches 0.95× its teacher's macro score (0.341 vs 0.358, against a measured null of 0.063), and with the best static teacher and a 60k vocabulary it routes 42 real corpus domains at 97% of its teacher's quality. Its argmax form stores a document in 80 bytes and still finds needles in a 1M-document haystack at 0.71 hit@1 and 0.92 hit@20. A routing rule built from the algebra itself, bundled centroids with a margin gate, routes open-set challenges at 92.4% precision while sending 86.9% of never-seen domains to spawn. Inside a verified reasoning pipeline, where a symbolic VM checks every hop, the table supports a claimed-answer precision of 0.992 at 0.646 coverage, unchanged under 100k distractors. Since August, the same word codes have also served as the address of an n-gram memory, where a trigram with one word paraphrased reaches its row 74.3% of the time against 4.0% for a different fact. We report the negative results with the same care, including a product-quantization win we retracted after finding codebook leakage, and the open-web entity search where the method ends.

1. Introduction

Two constraints collided in the CubbyLLM and MoWM architecture [1, 12]. First, the challenge path must contain no model: routing a challenge to a specialist world, deciding between route and spawn, and reading episodic memory happen on every input, and a sentence-encoder call of about 9 ms there dominates the whole budget. Second, the representation must support the VSA algebra, since worlds, axioms and episodic memory in this architecture are sparse block codes of $k = 80$ blocks by $l = 128$ slots, and whatever encodes text must land in that space or nothing composes. Random hypervectors satisfy the second constraint and fail semantics, while sentence

encoders have semantics and fail both. The standard VSA answer, which is hand-designed feature bindings, n-gram encoding or distributional learning inside the hypervector space, was screened here and measured to fall far short of a neural teacher (Sections 5.1 and 6).

Our observation is that the gap closes with a distillation, not a model. We embed a vocabulary once with the teacher, offline, clean its anisotropy, project it cosine-exactly into the block space and add back a random phasor identity per word. Serving is then a dictionary lookup plus an IDF-weighted sum, the oldest trick in information retrieval, running inside a representation that binds and bundles.

This paper gives a recipe that turns any text embedder into a qFHRR-compatible word table, with each design choice validated in an ablation arm, including the ones that failed. We show that argmax-discretized sparse block codes are qFHRR with integer phases, so the compact form of our table is at the same time an 80-byte storage format, an index scored without a fitted codebook, and a legal operand for phasor binding. We then measure a quality, latency and scale envelope across routing, temporal placement, retrieval and verified reasoning, we derive a routing rule from the algebra itself, and we report where the method stops working. The last section of results adds work done after the August version: the same word codes used as the address of an n-gram memory.

2. Background

Plate's holographic reduced representations in the frequency domain (FHRR) represent symbols as vectors of unit phasors [3]. Binding is element-wise complex multiplication, which is phase addition, bundling is complex summation, and unbinding is exact through conjugation. Sparse block codes, in the style of NVSA [4], split a D-dimensional vector into k blocks of length l with a single active slot per block, and binding becomes circular convolution per block, which for one-hot blocks reduces to modular addition of the active indices. Our substrate is the k = 80, l = 128 block algebra of `grilly` [5].

Snyder, Poursiami and Parsa's qFHRR quantizes FHRR phases to K discrete levels, which makes the whole algebra integer arithmetic [6]. A one-hot block of length 128 is a phasor with 128 quantized phases: the active index is the phase, and block-code binding (index addition mod l) is exactly qFHRR phasor multiplication. An earlier screen of ours measured K = 128 as already lossless at these dimensions, so quantization is not where quality is lost.

In every classical VSA construction the atomic vectors are random, so semantic similarity only exists where the modeler hand-encodes it. Learning it inside the hypervector space, as BEAGLE does with distributional memory [7], was measured here to smear the signal rather than sharpen it (Section 6). Meanwhile model2vec showed that a sentence encoder distills into a static per-token table at a modest quality cost [8], but into an unstructured dense space with no algebra. We take model2vec's distillation idea and land it in qFHRR space.

3. Method

3.1 Building the table

The teacher T is all-MiniLM-L6-v2 (384 dimensions) [9] in the first version. The vocabulary V is the top 30k words by document frequency over windows of the pretraining corpus, decoded through the project's BBPE-128k tokenizer, plus the domain axiom vocabulary, for 30,275 words.

We use words, not BPE tokens, because tokens as keys were measured consistently worse: subword fragments do not carry the teacher's lexical semantics the way words do. Each word is embedded once, offline, and unit-normalized, which takes about 5 minutes of CPU for the full vocabulary and never recurs. We then remove anisotropy by subtracting the mean vector and projecting out the first principal component, with the statistics fit on a broad external vocabulary rather than the evaluation corpus; Section 5.1 measures what that honest default costs.

A single orthonormal matrix Q, from the QR decomposition of a seeded Gaussian, maps 384 to $k \cdot l = 10{,}240$ dimensions. Orthonormality preserves inner products exactly, so no distortion enters at this step. Each word also gets a deterministic random block code, drawn with one active slot per block from a generator seeded by a BLAKE2b digest of the word. The stored vector is the 0.5/0.5 blend of the projected teacher vector and this identity, renormalized. The blend sharpens exact-word matching (+0.02 macro over pure distillation). It also gives unseen words a well-defined code at serve time through the same hash path, with no table entry needed. Per-word IDF weights from the same corpus statistics are stored alongside. The first shipped artifact is a 573 MB float16 file ($30{,}275 \times 80 \times 128$ plus IDF).

3.2 Serving, and two store forms

Encoding a text is a lowercase word split (words of two letters or more), a table lookup per word with the hash fallback for misses, an IDF-weighted sum and an L2 normalization. It is pure NumPy and takes 6.8 to 25 µs for short texts and about 0.5 to 1.1 ms for passages of about 700 characters. On the same CPU, that is 350× faster than the teacher on single texts and 14 to 30× faster than the teacher batched.

The same encoding has two store forms. The dense form (float16, 20 KB per document) keeps the full cosine geometry and is the quality ceiling. The compact form keeps only the argmax index of each block, 80 bytes per document, and that is the qFHRR integer-phase form. It is scored by gathering the dense query at each document's hot indices, which ranks exactly like cosine against the one-hot store, needs no fitted codebook and so carries no leakage risk, and remains a legal operand for phasor binding. Note that the dense stored vector is not a one-hot block code; it is a point in the same 80×128 space whose geometry carries the teacher's semantics, and the per-block argmax projects it onto the nearest legal qFHRR code. One table therefore gives dense vectors when the task is similarity and integer phasors when the task is storage, indexing or composition, and the two agree within the measured gap (hit@1 0.978 dense against 0.910 compact with no distractors, Section 5.4).

3.3 Routing with the algebra: centroids and margins

The VSA algebra has one superposition that preserves similarity (bundling) and one composition that destroys it (binding). An earlier milestone measured that a world's bind-chain signature is quasi-orthogonal to its own axioms, which makes it useless for routing. The routing rule that won our open-set screen uses each operation for what it preserves. The signature of each world is its axiom centroid, the normalized bundle of its axiom vectors, which needs one comparison per world (about 40× fewer than the maximum over axioms) and scores better (0.680 vs 0.589 macro). The gate routes to the best world only when its score clears an absolute floor and beats the second-best by a margin; otherwise the system spawns a new specialist, which is the recoverable outcome. Both thresholds are calibrated per deployment, and no threshold was observed to transfer across encoders or corpora.

4. Experimental discipline

All experiments are CPU-only standalone scripts with their logs kept (Windows 11, Python 3.12, NumPy 2.4.6). Three rules held throughout. Every screen runs its metrics over a pure-noise encoder or a matched analytic null, because comparisons based on a maximum have nulls far below 0.5, and this project caught five separate results compared against the wrong reference before the rule became mechanical. The teacher runs through the identical metric code in every screen, so "× teacher" is never arithmetic across papers. Finally, negative results are results (Section 6), including one retraction.

5. Results

5.1 The distillation screen

We route 280 held-out axiom texts to 15 domains by leave-one-out cosine; the matched macro null is 0.0633.

arm	macro	× null	µs per text
character 3-grams (baseline)	0.140	2.2	313
words + IDF (no semantics)	0.173	2.7	19
BPE token IDs + IDF	0.152	2.4	29
distilled tokens, PC1 removed, + IDF	0.204	3.2	28
distilled words, PC1 removed, + IDF	0.320	5.1	6.8
distilled words, PC1 removed, + signal blend (shipped)	**0.341**	**5.4**	25
MiniLM teacher	0.358	5.7	~9,000

The shipped arm is 0.95× the teacher, and every ingredient earns its place: words beat tokens by 0.12 macro at equal recipe, anisotropy removal is the largest single gain (+0.07), and the signal blend adds 0.02 plus the fallback for unseen words. When IDF and the principal component are refit on an external corpus of 20k documents, which is the deployment condition, the macro is 0.308, or 0.86× the teacher. The convenient in-domain statistics inflate the headline by about 0.03, so we report both and ship the external fit.

5.2 Routing at production scale, and the teacher ladder

On real corpus passages from 42 domains (40 exemplars and 40 challenges per domain, 28 closed-set domains scored and 5 held out as the open set), the first table routes at 0.595 macro against the batched teacher's 0.797. That is 0.75× the teacher at 13.7× its batched speed and 17.5× the measured noise null of 0.034, on passages far messier than the screen's axioms. The table's own build-time evaluation, with a different protocol, lands at 0.84× the teacher.

We then distilled every teacher we could through the identical recipe. The released potion-base-8M beats our MiniLM-taught table on 42-domain routing (0.722 vs 0.595 macro) but loses on placing New York Times abstracts in their year (0.044 vs 0.064 exact), and distilling potion itself into block space recovers routing to 0.646. The block-code target space costs nothing, since the projection is cosine-exact; the gap was teacher strength and vocabulary coverage.

teacher, distilled into block space	teacher dim	routing macro	NYT exact	NYT MAE median (years)
bge-m3 (GGUF Q8, direct)	1024	0.566	0.070	17
all-MiniLM-L6-v2 (v1 table)	384	0.595	0.064	19
potion-multilingual-128M	256	0.622	0.069	17
potion-base-8M (v2 table)	256	0.646	0.069	17
static-retrieval-mrl-en-v1	1024	0.670	**0.081**	15
potion-retrieval-32M (v3 table)	512	**0.680**	0.080	**15**
potion-base-8M, native (no block space)	256	*0.722*	*0.044*	*22*
all-MiniLM-L6-v2, live (the ceiling)	384	*0.797*	*0.046*	*21*

Raw bge-m3 [10] is a measured negative: a far stronger sentence embedder is a worse word-table teacher than MiniLM, because single words are not its input regime, while its compiled static form (potion-multilingual-128M, whose teacher is bge-m3) jumps to 0.622. The lesson is to pick teachers by the quality of their static token vectors, never by their rank on sentence benchmarks. Note also that every block-space table places years better than both native models, so the word, IDF and block recipe is the better serving form for temporal placement whatever the teacher.

Vocabulary was most of the remaining gap, and it saturates at 60k words. With the potion-retrieval-32M teacher, 30k words route at 0.680, 60k at 0.699 and 100k at 0.705 (within ±0.014 noise), against the teacher's native ceiling of 0.722. Meanwhile the table grows from 573 MB to 1.1 GB and 1.9 GB. The shipped v4 table (60,151 words, about 33 µs per challenge) sits at 97% of its teacher's routing quality, with the algebra and the 80-byte form intact.

5.3 Open-set routing and the cascade

Which score function best separates "routable to a known domain" from "never-seen domain, should spawn"? We screened 7 scorers over 3 rotated splits.

scorer	routed accuracy	spawn detection	AUC
maximum over exemplars, top-1	0.611	0.650	0.654
top-k mean, margin	0.886	0.842	0.657
centroid, margin	**0.924**	**0.869**	0.666
MiniLM (anchor)			0.825

The winner routes 68% of closed-set challenges with 92.4% precision and sends 86.9% of open-set challenges to spawn. It is the rule now wired into MoWM's bridge to CubbyLLM. With the v1 table, the gap in AUC to the teacher (0.666 vs 0.825) was the honest limit of a bag-of-words confidence signal. The teacher upgrade closed most of it: on the same split and statistic, the v4 table's closed-versus-open AUC is 0.690 against the teacher's 0.707.

We also measured a cascade where the teacher is consulted only inside an ambiguity band around the route-or-spawn boundary. The pure table is the most conservative and most precise profile, with 0.956 spawn detection and 0.935 routed precision at about 1,284 µs per challenge. The pure teacher gives 0.869 and 0.930 at about 17,546 µs. Escalating trades spawn detection down for routing coverage (0.355 of closed challenges routed by the table alone against 0.539 by the teacher)

at up to 13× the latency. The cascade is therefore a coverage lever for deployments that prefer routing over spawning, not a confidence fix.

5.4 Retrieval at scale

We asked 400 multi-hop questions with their 636 deduplicated supporting facts as targets and DBpedia abstracts as distractors, scoring whether any supporting fact comes first.

store size	dense (20 KB per document)	compact (80 bytes per document)
0	0.978	0.910
100k	0.973	0.853
1M	(too large for RAM)	0.713, hit@20 **0.920**

The teacher scores 0.993 with no distractors. The dense form is essentially flat to 100k distractors, and the compact form degrades gracefully, still holding 0.92 hit@20 at a million documents from 80 bytes each. Encoding the 1M store took 9 minutes (543 µs per document) against about 30 minutes for the batched teacher. The two-stage design follows directly: a compact shortlist of 20, then a dense re-rank of those 20. On a small science QA slice (60 pairs over 3 tags, directional only), the table's answer retrieval is 0.70× the teacher (hit@1 0.667 vs 0.95). Its domain routing, on the other hand, ties the teacher at 0.90 macro.

5.5 Fact injection and the multi-hop chase

On a fact-injection set of 10k questions, each shipping its supporting chain and one fact to inject, we stored 1,241 deduplicated facts. We then asked 800 questions (385, 256 and 159 at 1, 2 and 3 hops). After injection, the injected fact is in the top 5 for 99.5% of 1-hop questions but only 28.9% and 22.6% of 2- and 3-hop ones. The teacher sits on the same cliff (1.00, 0.34 and 0.30), because intermediate hops share only about 55% of their words with the question against about 75% for the first hop. Overall the table is again 0.95× the teacher (hit@5 0.616 vs 0.649), and 100k distractors move its injected-fact hit@1 from 0.481 to 0.480.

An iterative chase re-encodes the query with each retrieved fact appended, for as many rounds as there are hops, and it is still pure table lookups. It lifts the table's full-chain recovery from 0.665 to 0.733 overall and from 0.195 to 0.371 at 3 hops, about 1.9× on the same questions, and it holds 0.706 at 100k distractors. The identical protocol does nothing for the teacher (0.719 single-shot, 0.720 chased), which leaves the chased table at parity with the chased teacher. Query expansion is the natural multi-hop mechanism for a bag-of-words encoder, since each retrieved fact donates exactly the bridging words the question lacks, whereas appending text to a sentence-encoder query dilutes its embedding as much as it informs it.

5.6 Temporal worlds and inter-world query

We modeled time as worlds, each era and each year a world, routed by bundled centroid. On 4,000 curated dated events, the table routes eras at 0.669 macro (0.94× the teacher, null 0.163) and places events in their exact year at 0.735 over 239 year-worlds (0.98× the teacher). An arm with the digits masked gives the identical 0.735, so placement comes from the content, not from reading dates. Year-world centroids lose similarity steadily with temporal distance (0.342 under 10 years apart, 0.251 over 500 years apart). The idea that "the world was not the same from one year to

the next" is measurably in the representation. On 14,700 real New York Times abstracts over 147 year-worlds (1852 to 2024), the table beats the teacher on every year metric, with a median error of 19 years against 21 (null 50.7).

Each event's stated consequence is retrieved from among all 4,000 at 0.337 hit@1 and 0.631 hit@5 (0.75× the teacher; chance 0.00025). Consequence texts also route to later year-worlds than their events 55.2% of the time against a matched null of 44.6%, in both encoders. Gating the year choice behind an era gate costs about 19 points of exact-year accuracy, identically for the teacher, at 4× fewer comparisons. The repair is inter-world query, where a consulted world answers only if its best member clears a threshold and otherwise passes the query on. At a threshold of 0.6 it matches exhaustive flat search's answer rate with 63 member comparisons instead of 3,783. Two leakage variants were caught and excluded on the way: paraphrase-duplicate events inflating leave-one-out from 0.735 to 0.890, and full centroids letting a challenge's own vector nudge the ranking. As everywhere in this paper, the threshold is corpus-specific, and the 0.6 that delegates 2% of curated queries delegates about 99% of NYT queries.

5.7 Where the method ends

Open-web entity search with short keyword queries is the measured boundary. On BEIR's dbpedia-entity [11] with the full 4.64M-document corpus, the v1 table scores 0.040 nDCG@10 in compact form and 0.081 with the dense re-rank. The v4 table scores 0.046 and 0.096, against 0.224 for potion-8M dense and a published BM25 of about 0.313. Entity names are exactly the tail vocabulary that a word list built from document frequency misses, and the compact shortlist's recall@100 of about 0.14 caps the whole pipeline. The v4 rerun confirms the boundary is structural and not a teacher artifact: an 18% gain over v1, but no dent in the 2.3× gap to dense potion, which reproduced to four decimals across both runs. The table's validated domain is routing, temporal placement and retrieval over a known corpus, not open-vocabulary entity search.

5.8 The table inside a verified reasoning pipeline

The final test of the serving stack is a product property rather than a retrieval metric: can a pipeline built on the table give answers that are right when claimed? A grammar parses a multi-hop question into a relation chain (95.4% parse coverage after a six-frame grammar), and the table retrieves each hop's fact. A symbolic VM then holds the chain in a VSA frame, re-verifies every hop with a cosine confidence and reads the answer out, with a control role that was never bound in every program. Bound hops recover at about 0.50 similarity and the control at 0.029, a 17× margin.

The campaign took three rounds. The second one exposed a design error worth stating as a finding: the VM's similarity score measures binding fidelity, not truth. Planted near-miss faults bind wrong facts and recover them faithfully at high similarity, so semantic-hardness negatives cannot calibrate it. Round three calibrates what the score does defend, with floors conditional on frame size taken from correct-recovery quantiles, which land at 1.000, 0.474 and 0.220 for frames of 1, 2 and 3 pairs, the roughly 1/n bundle-dilution curve the algebra predicts. Under that calibration, at 0 and at 100k distractors with identical numbers, the pipeline reaches 0.646 verified coverage at 0.992 claimed-answer precision. It catches every planted fault in all four fault classes by symbol match, catches 0.77 to 0.80 of organic confusables, and passes 528 of 528 absent-role controls. The four verified-but-wrong answers are all collisions from the corpus's deduplication, where systematically misspelled surface forms collapse different chains' intermediate facts into one store entry. They

affect the unverified baselines identically.

This pipeline has since become the reasoning stack of CubbyLLM, described in our certified-execution preprint [12]. With a lookup-first walk and a plan checker in front of it, the same 800 questions now give 574 certified answers (0.718 coverage) at a precision of 1.000.

5.9 After August: an n-gram memory on the word codes

Qwen3.8-Flash-Next reads a 51B-parameter table of bigram and trigram embeddings from host RAM by direct address [13]. The main problem with a direct address is that a paraphrase misses it completely. We asked whether our word codes could address an n-gram memory that is exact like a hash for different facts but shared for paraphrases.

The first finding is a free one. At the shipped 0.5 blend, the per-block argmax of a word is its identity slot in most blocks. A trigram key built from the compact codes by qFHRR binding, with each word's phases rotated by position and added mod 128, is therefore a pure hash. Every pair class agrees at chance (1/128 per block), and the load over 597k distinct corpus trigrams is as uniform as a random hash's, with 24.85% of trigrams sharing a three-block bucket against 24.78%. The shipped table already gives a balanced, deterministic n-gram hash from 80-byte codes, decodable back to its words. On the other hand, the meaning lives in the teacher part. With the teacher part alone, semantic nearest neighbours agree on 24% of blocks (30× chance) and same-type entities such as *france* and *germany* on only 1.8%. However, exact buckets throw most of that away, and a soft multi-probe read over 26 tables recovers only part of it.

The read that works is a Hamming scan in the style of grilly's Rubik's-cube encoding. A trigram key is the bitwise majority of position role XOR word filler over 10,240 bits, with the filler taken as the sign of the word's table vector. A query is accepted when the original trigram is its nearest key within a distance cutoff. We swapped one word of each trigram and measured how often the original row is reached.

word codes (cutoff 0.15)	paraphrase reaches the row	a different fact reaches it	a random word
the shipped table (potion-retrieval-32M teacher)	22.3%	0%	0%
a paraphrase model's codes (paraphrase-MiniLM-L6-v2)	74.7%	10.7%	0%
the same, refined with WordNet (Attract-Repel)	**74.3%**	**4.0%**	0%

At a cutoff of 0.10 the refined table gives 31.3% against 0.3%, so one distance sets the trade-off between reaching a paraphrase and reading a different fact. The WordNet refinement [14, 15] trains one linear map on synonym, antonym and co-hyponym constraints with every test word banned from them. However, its repel weight was chosen on these same lists, so the pick still has to be confirmed on a fresh and larger list, and the paraphrase list itself is only 35 hand pairs. Note also that the n-gram memory and routing want different tables: the paraphrase teacher routes MoWM worlds at 0.246 against potion's 0.315, so routing keeps potion. The CPU scan takes about 330 ms per query over 597k keys; a GPU scan and the language-model test are next.

6. What did not work

Several things failed, and we report them with the same weight as the results. BEAGLE-style distributional memory, accumulating co-occurrence structure into hypervectors (raw, log or PPMI-weighted, over 200M tokens), scored below the signal-only baseline (0.064 to 0.083 macro against 0.113), as superposition smears the surface signal faster than co-occurrence adds semantics at this scale. Rotating codes by corpus frequency amplifies common grams and is strictly worse, complex-domain bundling is indistinguishable from argmax bundling, consistent with qFHRR being lossless here, and z-normalizing routing scores per domain gives nothing over raw cosine with one global threshold. BPE token IDs as table keys lose to words even without distillation (0.152 against 0.173), and bge-m3 as a direct word-table teacher routes at 0.566, below MiniLM's 0.595.

One result was retracted. An early run showed a product-quantization store outscoring the dense one, but the fitted codebook had seen the items that leave-one-out excluded. With the leak controlled, the win vanished. The compact form we ship uses no fitted codebook for exactly this reason. Product quantization remains viable when the number of axioms is much larger than $l \cdot k$, which is not our regime.

7. Limitations

The encoder is a weighted bag of words, so word order is lost; every n-gram and phase-binding variant we measured cost more than it bought at these scales, and tasks where order is the signal need the binding layer above the table. Thresholds do not travel: every pairing of encoder and corpus needs its own calibration, and we ship the screens as the calibration method, not the numbers. Unseen words get identity, not meaning, since the hash fallback makes them distinct but not semantic. All screens ran on a single machine, with 42 domains, 1M documents and 10k questions as the largest regimes, the science QA slice has only 60 pairs, and everything is English, lowercase, with words of two letters or more.

The multi-hop corpus is systematically misspelled, for example "Huamn" for "Human" in about 95% of its occurrences and "united stated" in 387 answers. The typos are consistent within each chain, so exact-match scoring is coherent. However, the accuracy figures of Sections 5.4, 5.5 and 5.8 measure chain-recovery fidelity against the corpus's own ground truth, not real-world correctness. An adversarial review of this paper also left three items open. The NYT dating result still needs a rerun with dates masked or scrubbed, a comparison against a teacher compressed to the table's byte budget has not been run, and the smaller evaluations still lack per-query bootstrap intervals.

8. Related work

The idea that VSA atoms need not be random has deep roots, and we join that lineage rather than found it. It includes Plate's HRR [3], Smolensky and Legendre's tensor-product binding with learned fillers [16], and Eliasmith's Semantic Pointer Architecture, in production in Spaun [17]. Random Indexing, where random index vectors accumulate semantics from co-occurrence, is the literal ancestor of our headline [18], and BEAGLE [7] belongs to the same line. Thomas, Dasgupta and Rosing show that the property that matters is incoherence, not randomness as such [19]. Semantic hashing established that learned discrete codes carry semantics [20].

On distillation, model2vec and potion distill sentence encoders into static token tables [8], and our first three steps follow their recipe; Sparse Distillation is the existence proof for the whole

approach, with n-gram-table students keeping about 97% of a RoBERTa classifier at 600× speedup [21]. Product quantization and asymmetric distance computation [22] inspired our compact scoring path, and our one-hot blocks make the codebook free and leakage-proof. The constrained-training line, such as JPQ [23] and finite scalar quantization [24], trains representations inside a quantized geometry instead of projecting afterwards, and headless language models [25] train against input-embedding targets with no softmax projection, the closest published neighbour of our retrieval-head economics. NVSA block codes [4] and qFHRR [6] supply the substrate and its integer-phase reading. Sparse distributed memory, modern Hopfield networks and resonator networks [26, 27, 28] are the associative-memory theory our delegation mechanisms inherit.

The bare claim that hypervectors can carry semantics is old. What we claim is the conjunction of teacher distillation, no neural encoder at serve time, a strongly structured qFHRR geometry, binding and bundling kept intact, and a strict byte budget. On top of it comes a measured envelope with quality ratios, scale curves, calibrated thresholds and negative results across routing, temporal placement, retrieval and verified reasoning. Random atoms are the right prior with no data, while distilled, structure-constrained atoms win when a teacher and a hard storage budget exist; quasi-orthogonality becomes a constraint preserved under learning rather than an axiom that requires randomness. Our own prior work used VSA transitions for a mixture of world models [1] and learnable reasoning rules on hypervectors [2].

9. Conclusion and open problems

VSA atoms do not have to trade semantic geometry for algebraic structure. A teacher distilled through a cosine-exact projection puts real semantic geometry into qFHRR phasor space at 0.75 to 0.98× the teacher's measured quality. With the best static teacher and a saturated vocabulary, it reaches 0.97×. A 0.5/0.5 blend with hashed phasor identities keeps exact-match sharpness and coverage of unseen words, and the per-block argmax turns any encoding into an 80-byte integer-phase code that scores at millions of documents and binds like any other block code. The serving path is a dictionary and a matrix product, and it is strong enough to be the retriever of a verified reasoning pipeline. For architectures that route, spawn, remember and prove on every input, and cannot afford a model call to do it, that is the property that matters.

This work leaves six open problems, none of which we found solved in published work. The first is weighted bundling as an amortized Bayesian posterior, with evidence-cue unbinding as the likelihood and a reversible, temperature-calibrated collapse as the selection. The second is training directly inside sparse block codes, against our train-free-then-project baseline, and the third is debate between heterogeneous specialist stores that propose only machine-checkable claims judged by deterministic execution. The fourth is telling absence apart from misrouting in partitioned multi-hop retrieval, and the fifth is language modeling with both vocabulary surfaces, input table and retrieval index, distilled from one teacher into a shared geometry. The last is a clean ablation of retrieval in place of attention in hybrid recurrent language models. The n-gram memory of Section 5.9 is a first step on the last one.

Appendix A. Reproducibility map

Scripts and logs are kept in GRL's research repository; `grilly` is open source [5].

claim	script	log
screen arms and the OOD check (5.1)	`exp_f2m2_semantic_routing.py` (`--fast-arms`, `--m2v`, `--m2v-ood`)	`exp_f2m2_fast_arms`, `exp_f2m2_m2v`, `exp_f2m2_m2v_ood`
BEAGLE negative (6)	`exp_f2m2_semantic_routing.py --beagle`	`exp_f2m2_beagle`
42-domain routing and open set (5.2, 5.3)	`exp_m3_domain_routing.py` (`--open-screen`)	`exp_m3_domain_routing`, `exp_m3_open_set`
potion baseline, blend, teacher ladder (5.2)	`exp_m3_potion_baseline.py`, `exp_m3_teacher_ladder.py`	`exp_m3_potion_baseline`, `exp_m3_teacher_ladder`, `exp_m3_bgem3_block_arm`
vocabulary scaling (5.2)	`build_fastword_table.py --top-words`	`exp_m3_table_{v2,60k,100k}_validation`
cascade (5.3)	`exp_m3_cascade.py`	`exp_m3_cascade`
science QA and haystack (5.4)	`exp_m3_science_qa.py`, `exp_m3_haystack.py`	`exp_m3_science_qa`, `exp_m3_haystack`
injection and chase (5.5)	`exp_m3_injection.py`	`exp_m3_injection`
temporal worlds and NYT years (5.6)	`exp_m3_temporal_causal.py`, `exp_m3_nyt_years.py`	`exp_m3_temporal_causal`, `exp_m3_nyt_years`
BEIR dbpedia-entity (5.7)	`exp_m3_beir_dbpedia.py`	`exp_m3_beir_dbpedia`
verified pipeline (5.8)	`exp_m3_cot_pipeline.py`	`exp_m3_cot_pipeline*`
n-gram memory (5.9)	`exp_hm1_ngram_hv.py`, `exp_hm1b_soft_address.py`, `exp_hm1c_hamming.py`, `hm1_attract_repel.py`	hypotheses file, H-M1 entries

References

1. Cloutier, N. MoWM: Mixture of World Models via Hypernetwork-Generated VSA Transitions. Zenodo, March 2026.
2. Cloutier, N. Learnable Reasoning Rules for Neuro-Vector-Symbolic Architectures via Hidden Markov Models. Zenodo, March 2026.
3. Plate, T. A. Holographic Reduced Representations. IEEE Transactions on Neural Networks 6(3), 1995.
4. Hersche, M. et al. A neuro-vector-symbolic architecture for solving Raven's progressive matrices. Nature Machine Intelligence, 2023.
5. Grilly, a Vulkan compute framework. github.com/Grillcheese-AI/grilly.
6. Snyder, Poursiami and Parsa. qFHRR: Rethinking Fourier Holographic Reduced Representations through Quantized Phase and Integer Arithmetic. arXiv:2604.25939, 2026.
7. Jones, M. N. and Mewhort, D. J. K. Representing word meaning and order information in a composite holographic lexicon. Psychological Review 114(1), 2007.
8. Tulkens, S. and van Dongen, T. Model2Vec. MinishLab, 2024. github.com/MinishLab/model2vec.
9. Reimers, N. and Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
10. Chen, J. et al. BGE M3-Embedding. arXiv:2402.03216, 2024.

11. Thakur, N. et al. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS 2021 Datasets and Benchmarks.
12. Cloutier, N. From probabilistic Chain-of-Thoughts to Certified Execution. Preprint, Zenodo, doi:10.5281/zenodo.24020073, 2026.
13. Qwen Team. Qwen3.8-Flash-Next, an experimental preview. August 2026.
14. Mrkšić, N. et al. Semantic Specialization of Distributional Word Vector Spaces using Monolingual and Cross-Lingual Constraints (Attract-Repel). TACL 2017.
15. Miller, G. A. WordNet: A Lexical Database for English. Communications of the ACM 38(11), 1995.
16. Smolensky, P. and Legendre, G. The Harmonic Mind. MIT Press, 2006.
17. Eliasmith, C. et al. A Large-Scale Model of the Functioning Brain. Science 338, 2012.
18. Sahlgren, M. An Introduction to Random Indexing. TKE workshop, 2005.
19. Thomas, A., Dasgupta, S. and Rosing, T. A Theoretical Perspective on Hyperdimensional Computing. JAIR 72, 2021.
20. Salakhutdinov, R. and Hinton, G. Semantic Hashing. International Journal of Approximate Reasoning 50(7), 2009.
21. Ye, Q. et al. Sparse Distillation: Speeding Up Text Classification by Using Bigger Student Models. NAACL 2022.
22. Jégou, H., Douze, M. and Schmid, C. Product Quantization for Nearest Neighbor Search. IEEE TPAMI 33(1), 2011.
23. Zhan, J. et al. Jointly Optimizing Query Encoder and Product Quantization to Improve Retrieval Performance (JPQ). CIKM 2021.
24. Mentzer, F. et al. Finite Scalar Quantization: VQ-VAE Made Simple. ICLR 2024.
25. Godey, N. et al. Headless Language Models: Learning without Predicting with Contrastive Weight Tying. ICLR 2024.
26. Kanerva, P. Sparse Distributed Memory. MIT Press, 1988.
27. Ramsauer, H. et al. Hopfield Networks is All You Need. ICLR 2021.
28. Frady, E. P., Kent, S. J., Olshausen, B. A. and Sommer, F. T. Resonator Networks. Neural Computation 32(12), 2020.